TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero-Agosto 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA Y SERIE:
PATRONES DE DISEÑO

TÍTULO ACTIVIDAD:
Actividad de valor en equipo

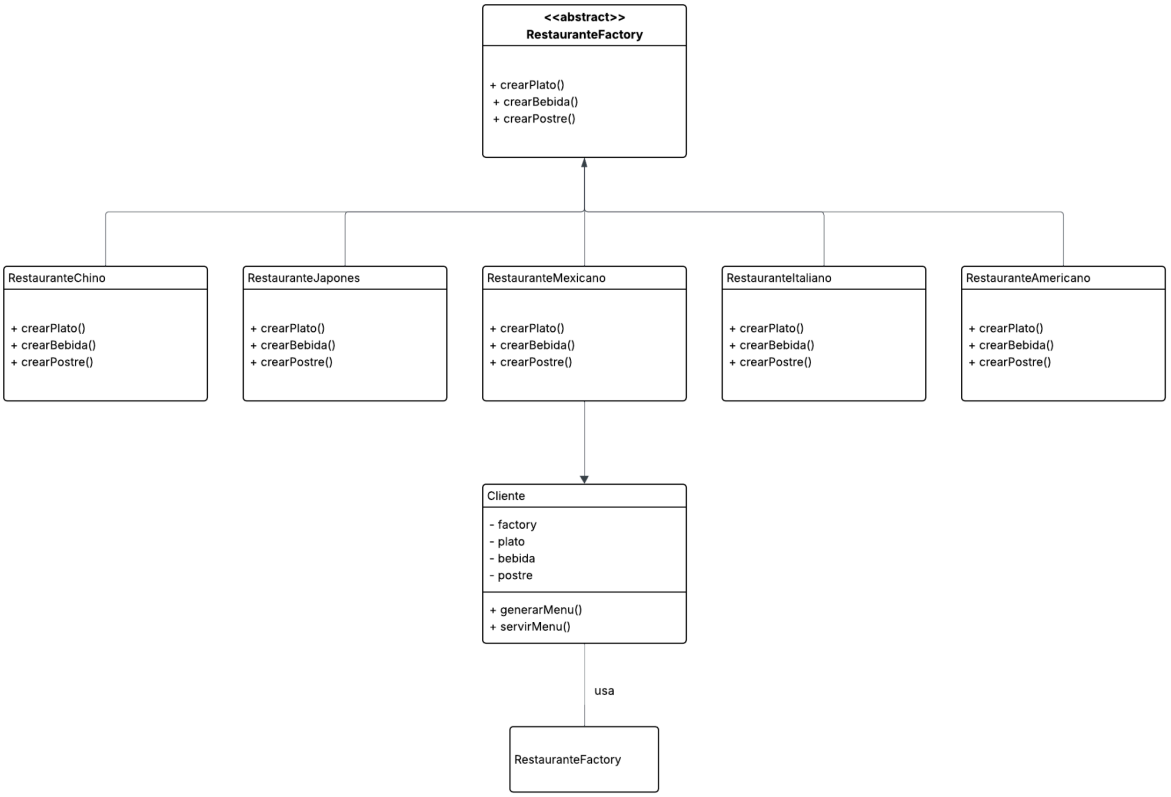
UNIDAD A EVALUAR:
2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
MENDOZA SUAREZ IVAN GUSTAVO 22210910
FELIX SANDOVAL JOSUE GUADALUPE 21211940
ORTEGA RAMOS YAHIR 21212578
SOTELO RUBIO RODRIGO 21212053

NOMBRE DEL MAESTRO (A):
MARIBEL GUERRERO LUIS

Índice

Diagrama UML



En la parte superior está la clase RestauranteFactory, que es una clase abstracta donde se definen los métodos crearPlato(), crearBebida() y crearPostre(). Después aparecen las clases RestauranteChino, RestauranteJapones, RestauranteMexicano, RestauranteItaliano y RestauranteAmericano, las cuales heredan de la fábrica abstracta y se encargan de crear su propio menú dependiendo del tipo de restaurante. Finalmente está la clase Cliente, que utiliza la fábrica para generar el menú sin depender de un restaurante específico. Esto permite cambiar fácilmente entre diferentes restaurantes sin modificar el código del cliente, que es la idea principal del patrón Abstract Factory.

Código HTML

```
<!DOCTYPE html>

<html lang="es">

<head>

<meta charset="UTF-8">

<title>Abstract Factory - Restaurantes del Mundo</title>

<link rel="stylesheet" href="styles.css">

</head>

<body>

<header>

<h1>Restaurantes del Mundo</h1>

<p>Seleccione una cocina para generar el menú</p>

</header>

<div class="selector">

<select id="restaurante">

<option value="chino">Restaurante Chino</option>

<option value="japones">Restaurante Japonés</option>

<option value="mexicano">Restaurante Mexicano</option>

<option value="italiano">Restaurante Italiano</option>

<option value="americano">Restaurante Americano</option>

 </select>

<button onclick="crearCliente()">Generar Menú</button>
```

```
</div>

<div class="menu-container">

<div class="card">

<img id="imgPlato">

<h3 id="plato"></h3>

</div>

<div class="card">

<img id="imgBebida">

<h3 id="bebida"></h3>

</div>

<div class="card">

<img id="imgPostre">

<h3 id="postre"></h3>

</div>

</div>

<script src="script.js"></script>

</body>

</html>
```

Código JS

```
// FABRICA ABSTRACTA

class RestauranteFactory{

crearPlato(){}

crearBebida(){}

crearPostre(){}

}

// RESTAURANTE CHINO

class RestauranteChino extends RestauranteFactory{

crearPlato(){

return {nombre:"Chow Mein",img:"ChowMein.jpg"};

}

crearBebida(){

return {nombre:"Té Jazmín",img:"TeJazmín.jpg"};

}

crearPostre(){

return {nombre:"Rollito dulce",img:"RollitoDulce.jpg"};

}

}

// RESTAURANTE JAPONES

class RestauranteJapones extends RestauranteFactory{

crearPlato(){

return {nombre:"Ramen",img:"https://images.unsplash.com/photo-1557872943-16a5ac26437e"};
```

```
}

crearBebida(){

return {nombre:"Sake",img:"https://images.unsplash.com/photo-1597075687491-cf2c9c21a3c6"};

}

crearPostre(){

return
{nombre:"Dango",img:"https://images.unsplash.com/photo-1608032077018-c9aad9565d29"};

}



}

// RESTAURANTE MEXICANO

class RestauranteMexicano extends RestauranteFactory{

crearPlato(){

return {nombre:"Tacos de carne
asada",img:"https://images.unsplash.com/photo-1551504734-5ee1c4a1479b"};

}

crearBebida(){

return {nombre:"Agua de
Jamaica",img:"https://images.unsplash.com/photo-1598135753163-6167c1a1ad65"};

}

crearPostre(){

return {nombre:"Pastel tres
leches",img:"https://images.unsplash.com/photo-1621303837174-89787a7d4729"};

}

}
```

```
// RESTAURANTE ITALIANO

class RestauranteItaliano extends RestauranteFactory{

crearPlato(){

return
{nombre:"Lasagna",img:"https://images.unsplash.com/photo-1619895092538-128341789043"};

}

crearBebida(){

return {nombre:"Vino
tinto",img:"https://images.unsplash.com/photo-1506377247377-2a5b3b417ebb"};

}

crearPostre(){

return
{nombre:"Tiramisú",img:"https://images.unsplash.com/photo-1571877227200-a0d98ea607e9"};

}

}

// RESTAURANTE AMERICANO

class RestauranteAmericano extends RestauranteFactory{

crearPlato(){

return
{nombre:"Hamburguesa",img:"https://images.unsplash.com/photo-1568901346375-23c9450c58c
d"};

}

crearBebida(){

return
{nombre:"Refresco",img:"https://images.unsplash.com/photo-1581636625402-29b2a704ef13"};

}
```

```
crearPostre(){

return
{nombre:"Cheesecake",img:"https://images.unsplash.com/photo-1563805042-7684c019e1cb"};

}

}

// CLIENTE

class Cliente{

constructor(factory){

this.factory = factory;

}

generarMenu(){


this.plato = this.factory.crearPlato();

this.bebida = this.factory.crearBebida();

this.postre = this.factory.crearPostre();

}

servirMenu(){

document.getElementById("plato").innerText=this.plato.nombre;

document.getElementById("bebida").innerText=this.bebida.nombre;

document.getElementById("postre").innerText=this.postre.nombre;

document.getElementById("imgPlato").src=this.plato.img;

document.getElementById("imgBebida").src=this.bebida.img;

document.getElementById("imgPostre").src=this.postre.img;
```

```
}

}

// CAMBIO DE FABRICA

function crearCliente(){

let tipo=document.getElementById("restaurante").value;

let factory;

if(tipo=="chino") factory=new RestauranteChino();

else if(tipo=="japones") factory=new RestauranteJapones();

else if(tipo=="mexicano") factory=new RestauranteMexicano();

else if(tipo=="italiano") factory=new RestauranteItaliano();

else factory=new RestauranteAmericano();

let cliente=new Cliente(factory);

cliente.generarMenu();

cliente.servirMenu();

}
```

Código CSS

```
body{

font-family: 'Segoe UI', sans-serif;
margin:0;
background:#f5f5f5;
text-align:center;

}

header{

background:#222;
color:white;
padding:30px;

}

.selector{

margin:20px;

}
```

```
select, button{

padding:10px;
font-size:16px;
border-radius:6px;
border:none;
margin:5px;

}

button{

background:#ff6b6b;
color:white;
cursor:pointer;

}

button:hover{

background:#ff3b3b;
```

```
}

.menu-container{

display:flex;
justify-content:center;
gap:25px;
flex-wrap:wrap;
margin-top:30px;

}

.card{

background:white;
width:250px;
border-radius:10px;
box-shadow:0 4px 12px rgba(0,0,0,0.2);
padding:15px;
transition:0.3s;

}
```

```
.card:hover{

transform:scale(1.05);

}

.card img{

width:100%;
height:160px;
object-fit:cover;
border-radius:8px;

}

h3{

margin-top:10px;

}

.card img{
width:100%;
```

```
height:160px;

object-fit:cover;

border-radius:10px;

}
```

¿Qué es el patrón utilizado?

El patrón Abstract Factory permite crear objetos de una familia relacionada (en este caso, platos, bebidas y postres de diferentes cocinas) a través de una interfaz común, sin que el cliente conozca las clases concretas. En el código:

- RestauranteFactory es la fábrica abstracta, que define los métodos crearPlato(), crearBebida() y crearPostre().
- Las clases concretas como RestauranteChino, RestauranteJapones, etc., extienden RestauranteFactory e implementan estos métodos para crear objetos específicos de cada cocina (por ejemplo, un plato chino es "Chow Mein", mientras que uno japonés es "Ramen").
- La clase Cliente usa la fábrica para generar y servir el menú, sin saber qué tipo de restaurante está creando los objetos. Esto promueve el principio de inversión de dependencias y facilita la extensión del código.

Este patrón es ideal para escenarios donde necesitas crear productos relacionados (como menús completos de una cocina) y quieres evitar acoplamientos directos entre el cliente y las clases concretas.

¿Cómo se evita el usar tanto NEW al utilizar este patrón?

En el código original, sin el patrón, el cliente tendría que usar new directamente para crear cada objeto individual (por ejemplo, new ChowMein(), new TeJazmin(), etc.), lo que genera acoplamiento fuerte y hace difícil cambiar o agregar nuevas cocinas sin modificar el cliente. Con Abstract Factory:

- El cliente (Cliente) no usa new para crear los objetos concretos. En su lugar, delega la creación a la fábrica abstracta (this.factory.crearPlato(), etc.).
- La única instancia de new en el cliente está en la función crearCliente(), donde se selecciona la fábrica concreta basada en el tipo de restaurante (por ejemplo, new RestauranteChino()). Esto centraliza la lógica de creación y reduce el uso de new en el resto del código.
- Si no se usara el patrón, cada vez que se agregara un nuevo producto (como un plato), habría que modificar el cliente para instanciarlo con new, lo que viola el principio abierto-cerrado. Aquí, el patrón encapsula la creación, permitiendo que el cliente sea independiente de las clases concretas.

¿Cómo agregar una nueva cocina?

Primero, se crea una nueva clase concreta de fábrica en el archivo script.js. Por ejemplo, si se quiere agregar un "Restaurante Francés", se escribe una clase que extienda RestauranteFactory y defina los métodos para crear el plato, la bebida y el postre específicos de esa cocina.

Algo así:

```
// RESTAURANTE FRANCÉS

class RestauranteFrances extends RestauranteFactory {

    crearPlato() {

        return { nombre: "Boeuf Bourguignon", img: "https://example.com/imagen-plato.jpg" };

    }

    crearBebida() {

        return { nombre: "Vino Bordeaux", img: "https://example.com/imagen-bebida.jpg" };

    }

    crearPostre() {

        return { nombre: "Crème Brûlée", img: "https://example.com/imagen-postre.jpg" };

    }

}
```

Se eligen nombres y URLs de imágenes que representen bien la cocina francesa.

Después, se actualiza el archivo index.html para agregar la nueva opción en el menú desplegable. En la etiqueta <select> con id="restaurante", se añade una nueva línea de opción, como esta:

```
<option value="frances">Restaurante Francés</option>
```

Esto se pone justo después de las opciones existentes, para que aparezca en la lista cuando el usuario seleccione.

Finalmente, se modifica la función crearCliente() en script.js para manejar la nueva selección. En la parte donde se evalúa el tipo del restaurante, se añade un nuevo else if para crear la fábrica correspondiente. Quedaría algo así:

```
else if (tipo === "frances") factory = new RestauranteFrances();
```

Esto va justo antes del último else que maneja el restaurante americano.

Con estos cambios, ya se puede seleccionar "Restaurante Francés" en la interfaz, hacer clic en "Generar Menú" y ver el menú completo generado automáticamente.

Conclusiones

Felix Sandoval Josue Guadalupe 21211940

En conclusión, este proyecto demuestra cómo un enfoque inteligente facilita la adición de nuevas cocinas de manera eficiente, sin complicar el código. Me hace reflexionar sobre la importancia de planificar para el futuro en cualquier proyecto, y me motiva a aplicar ideas similares en mi trabajo diario.

Mendoza Suarez Ivan Gustavo

El propósito del patrón es crear diferentes grupos de objetos relacionados usando una misma estructura, sin que el cliente tenga que saber qué restaurante específico se está utilizando. Esto hace que el programa sea más organizado, flexible y fácil de ampliar. Si no se utilizara este patrón, el código tendría muchos if o condiciones para crear cada tipo de objeto, lo que lo volvería más desordenado y difícil de mantener o modificar. Por eso, usar este patrón ayuda a mantener el código más limpio y escalable.

Ortega Ramos Yahir

En este trabajo utilizamos el patrón de diseño Abstract Factory para organizar la creación de los diferentes menús de los restaurantes. Gracias a este patrón, puede generar fácilmente platos, bebidas y postres de distintos tipos de restaurantes (chino, japonés, mexicano, italiano y americano) sin tener que escribir código repetido. Cada tipo de restaurante tiene su propia “fábrica” que define los productos que ofrece, y el cliente simplemente usa la fábrica para generar su menú.

Esto hace que el código sea más ordenado, flexible y fácil de mantener, porque si quiero agregar un nuevo restaurante o cambiar algún plato, solo modificó su fábrica correspondiente sin afectar el resto del sistema. Además, con la funcionalidad del carrito y el modal para mostrar el ticket, logré que la experiencia del usuario sea más interactiva y profesional, manteniendo todo dentro de la misma página y evitando cargas innecesarias.

Sotelo Rubio Rodrigo

En este proyecto se aplicó el patrón de diseño Abstract Factory Pattern para estructurar la creación de diferentes menús de restaurantes de forma organizada. Gracias a este enfoque, el sistema permite generar objetos relacionados como platos, bebidas y postres sin depender directamente de clases concretas, lo que reduce el acoplamiento del código.

Una de las principales ventajas que observé es que este patrón facilita la escalabilidad del sistema. Si en el futuro se desea agregar un nuevo tipo de restaurante o modificar algún elemento del menú, únicamente es necesario crear una nueva fábrica o modificar la existente sin afectar el resto del programa. Esto ayuda a mantener el código más limpio, modular y fácil de mantener.